> **NOTE** In case you're not familiar with the Log4j challenges I mention here (or maybe you've forgotten about them), CVE-2021-44228 (https://nvd.nist.gov/vuln/detail/CVE-2021-44228), informally known as Log4Shell, was a zero-day vulnerability in Log4j, discovered in November 2021. It caused a lot of work (to apply the security patches) in a lot of places, from internal IT to cloud providers, and serves as an example of how much operational impact such a thing can have.

While logs are stable from an API and model perspective, in OpenTelemetry it may still take until mid-2024 until they are supported throughout in the SDKs and distributions. This means you will need a temporary strategy until you can, eventually, migrate to OpenTelemetry.

OK, that's enough theory. Let's see log routing in action. First up is a popular choice in the context of containers: Fluentd and Fluent Bit.

4.1.1 Fluentd and Fluent Bit

The CNCF projects Fluentd (https://www.fluentd.org/) and Fluent Bit (https://fluentbit.io/) are popular cloud-native agents that started out in the logging domain. Initially, there was only Fluentd (written in Ruby and C), offering a rich ecosystem (with over 1,000 plug-ins) but coming with a relatively big memory footprint (dozens of MB) for the agent.

In 2014, the folks behind Fluentd, Treasure Data, acknowledged the need for a lightweight log router specifically for constraint environments (containers, IoT). This insight led to the development of Fluent Bit (written in C) with a memory footprint of a couple of hundreds of KB and many fewer plug-ins than Fluentd (less than 100).

At time of writing, Fluent Bit is expanding its scope from logs-only to other signal types, such as metrics and traces. For details, have a look at the GitHub issue "Fluent Bit v2.0 Is Coming" (https://github.com/fluent/fluent-bit/discussions/5993). We will focus on the logging aspects, in this section, however, since there are still many WIP items in flight. Let's have a look at a basic usage example in a containerized setup and discuss some properties along the way.

To demonstrate log routing, we're using an OpenSearch with Docker Compose (https://opensearch.org/docs/latest/opensearch/install/docker/) setup. Don't worry about OpenSearch for now; we will cover it in detail in chapter 5. Just think of it as the place logs can be sent to and which you can use to look up, search for, and query log lines. For example, you may be interested in all `ERROR`-level logs lines from a specific microservice, within a certain time range (say, the last 24 hours).

In figure 4.1, you can see our example logging pipeline—we're using `hollу` (http://mng.bz/rWxg) as the log signal source running in a Docker container. The container is exposing port 8765 for its HTTP API and uses the Fluentd logging driver to send log lines to Fluent Bit (the main log router), ingesting log lines into OpenSearch as the log backend destination, which you then can query and visualize in OpenSearch Dashboards.